

Arquitetura Multicamadas para Segurança em Modelos de Linguagem de Grande Escala (LLMs)

Nome Sobrenome, *Member, IEEE*, Nome Sobrenome, *Member, IEEE*

Abstract—Este artigo apresenta uma arquitetura multicamadas para segurança em Modelos de Linguagem de Grande Escala (LLMs), integrando sanitização, detecção de ameaças, mitigação de vieses e validação pós-geração. Os experimentos conduzidos demonstram alta eficácia contra ataques explícitos e desafios persistentes na detecção de vieses linguísticos, contribuindo para avanços em segurança e governança de IA.

Index Terms—Segurança em IA, LLMs, Prompt Injection, Guardrails, Arquitetura de Segurança, Bias Detection

I. INTRODUÇÃO

Modelos de Linguagem de Grande Escala (LLMs) têm se tornado componentes centrais em aplicações modernas, incluindo assistentes virtuais, sistemas conversacionais, plataformas educacionais e ambientes corporativos. Sua capacidade de gerar, interpretar e transformar linguagem natural em larga escala permite níveis de automação antes inviáveis. Entretanto, essa mesma flexibilidade introduz riscos significativos relacionados à segurança, ética e confiabilidade, especialmente quando tais modelos são expostos diretamente a usuários finais ou integrados a sistemas críticos.

Diversas vulnerabilidades inerentes aos LLMs têm sido amplamente documentadas na literatura, incluindo ataques de *prompt injection*, tentativas de *jailbreak* para subverter instruções do sistema, geração de conteúdo malicioso, vieses socioculturais reproduzidos ou amplificados pelo modelo e alucinações (*hallucinations*) que podem resultar em informações factualmente incorretas ou perigosas. Além disso, LLMs podem inadvertidamente vaziar informações pessoais identificáveis (PII), expondo organizações e usuários a riscos legais e de privacidade.

Soluções tradicionais baseadas apenas em filtragem estática, listas de bloqueio ou simples verificações sintáticas têm se mostrado insuficientes para lidar com ameaças cada vez mais sofisticadas. Isso ocorre porque muitos ataques exploram dependências semânticas, ambiguidades linguísticas e lacunas nos mecanismos de alinhamento do modelo, exigindo abordagens mais abrangentes e sensíveis ao contexto.

Diante desse cenário, este trabalho propõe uma arquitetura multicamadas para segurança de LLMs, composta por módulos de sanitização estrutural, detecção de ameaças explícitas, identificação de vieses e validação de saídas, orquestrados de forma sistemática por um serviço centralizado. O objetivo é oferecer um mecanismo de defesa capaz de atuar em diferentes níveis do fluxo de interação, mitigando riscos antes e depois da geração da resposta pelo modelo.

Para avaliar a eficácia da solução proposta, foram conduzidos experimentos envolvendo 30 casos de teste distribuídos

entre categorias críticas, incluindo ataques maliciosos, tentativas de *jailbreak*, exposição de PII e múltiplas subcategorias de vieses linguísticos. Métricas quantitativas — como precisão, recall, acurácia e F1-score — foram utilizadas para mensurar o desempenho da arquitetura em cenários adversariais. Os resultados indicam desempenho superior nas categorias de risco explícito, enquanto apontam desafios persistentes na detecção de vieses implícitos.

As principais contribuições deste trabalho são:

- a proposição de uma arquitetura multicamadas para segurança em LLMs, combinando verificações sintáticas, estruturais e semânticas;
- o desenvolvimento de módulos especializados para sanitização, bloqueio de ataques, detecção de vieses e validação pós-geração;
- a definição de um fluxo padronizado de comunicação que aumenta a interpretabilidade e auditabilidade das decisões de segurança;
- a avaliação experimental da arquitetura em um conjunto diversificado de cenários adversariais, com análise quantitativa e comportamental dos resultados.

O restante deste artigo está estruturado da seguinte forma. A Seção II apresenta os fundamentos teóricos relacionados a LLMs e mecanismos de segurança. A Seção III discute os principais trabalhos relacionados. A Seção IV descreve a arquitetura do sistema. A Seção V apresenta o modelo conceitual subjacente. A Seção VI discute os resultados experimentais. Por fim, a Seção VII apresenta as conclusões e trabalhos futuros.

II. METODOLOGIA

A metodologia adotada neste trabalho foi estruturada de forma a avaliar, de maneira sistemática e controlada, a eficácia da arquitetura proposta para segurança em Modelos de Linguagem de Grande Escala (LLMs). O processo metodológico integra a geração de cenários de ataque, a classificação de riscos, a execução de testes em múltiplos idiomas e a análise quantitativa dos resultados, permitindo uma investigação abrangente das capacidades e limitações do sistema.

A. Estratégia Experimental

O experimento foi concebido como um *ataque exploratório contínuo*, cujo objetivo é simular o comportamento de um agente malicioso persistente. Essa abordagem consiste na geração automática de entradas projetadas para induzir o LLM a violar políticas de segurança, produzir conteúdo nocivo, revelar informações sensíveis ou demonstrar vieses linguísticos. A estratégia busca imitar situações realistas, considerando que

agentes adversários frequentemente exploram múltiplas técnicas, idiomas e estruturas linguísticas para burlar mecanismos de defesa.

A escolha por uma metodologia exploratória contínua permitiu avaliar não apenas casos isolados, mas também a capacidade do sistema de manter níveis consistentes de segurança diante de ataques sucessivos.

B. Geração Multilíngue de Cenários

Um dos diferenciais desta metodologia é o uso de cenários de ataque em múltiplos idiomas, incluindo português, inglês e espanhol. A literatura demonstra que diversos ataques de *prompt injection* e *jailbreak* apresentam variações linguísticas significativas, sendo possível contornar filtros monolíngues por meio de:

- mudanças de idioma;
- alternância entre códigos linguísticos (*code-switching*);
- tradução contextual com semântica preservada;
- inserção de ambiguidades culturais específicas.

Ao incluir múltiplos idiomas, o experimento adotou uma perspectiva mais realista e alinhada ao contexto global de uso dos LLMs, ampliando a cobertura metodológica e expondo limitações que dificilmente seriam observadas em cenários monolíngues. A literatura existente ainda não explora de forma ampla essa dimensão, reforçando a originalidade da abordagem.

C. Construção dos Casos de Teste

Foram desenvolvidos 30 casos de teste distribuídos entre quatro grupos principais:

- **Ataques explícitos:** tentativas de *prompt injection*, indução maliciosa e manipulação direta do comportamento do modelo;
- **Deteção de PII:** solicitações envolvendo CPF, e-mail, telefone, cartão de crédito e número de seguridade social;
- **Vieses linguísticos:** categorias relacionadas a gênero, raça, idade, corpo, orientação, localização geográfica e educação;
- **Alucinações:** prompts projetados para induzir respostas factualmente incorretas ou inconsistentes.

Cada caso de teste foi submetido à arquitetura completa, contemplando sanitização, verificação de entrada, processamento pelo LLM e auditoria na saída.

D. Métricas de Avaliação

As seguintes métricas foram utilizadas para análise quantitativa dos resultados:

- **Precisão (*Precision*)** — mede a proporção de classificações corretas entre as respostas identificadas como risco;
- **Revocação (*Recall*)** — indica a capacidade do sistema de detectar todos os riscos reais;
- **F1-score** — combinação harmônica entre precisão e recall, útil em cenários com desequilíbrio entre classes;
- **Acurácia** — porcentagem geral de classificações corretas;

- **Taxa de sucesso** — percentual de casos em que a ameaça foi devidamente identificada e bloqueada;
- **Tempo de resposta** — medição do custo computacional associado a cada módulo.

Essas métricas permitem identificar tanto o desempenho geral quanto a sensibilidade do sistema a categorias específicas de ameaças.

E. Pipeline de Execução

O pipeline experimental segue cinco etapas principais:

- 1) Geração automática ou manual dos prompts adversariais.
- 2) Execução do fluxo completo da arquitetura (sanitização, verificação, LLM e auditoria de saída).
- 3) Registro estruturado dos resultados, incluindo classificação, métricas e tempos de resposta.
- 4) Cálculo das métricas por categoria e comparação entre classes.
- 5) Análise dos dados, interpretação dos padrões e identificação de limitações.

O uso de um pipeline padronizado permite a reprodutibilidade dos experimentos e facilita auditorias futuras.

F. Resumo

De forma geral, a metodologia adotada incorpora elementos exploratórios, linguísticos, quantitativos e comportamentais, proporcionando uma avaliação abrangente da robustez da arquitetura. A abordagem multilíngue, a simulação contínua de ataques e o uso de métricas rigorosas ampliam significativamente a validade dos resultados apresentados na Seção VI.

III. TRABALHOS RELACIONADOS

TABLE I
COMPARISON OF RELATED WORKS ON PROMPT INJECTION DEFENSE IN LLMs

Referencia	Foco	Metodologia	Métricas	Lacuna
Lin, Huawei <i>et al.</i> [1]	Proposta de defesa unificada (UniGuardian) contra ataques de LLM (injeção, backdoor, adversariais).	Propõe um framework de detecção unificado para analisar prompts e saídas.	(1) auROC (Area Under the Receiver Operator Characteristic Curve) e (2) auPRC (Area Under the Precision-Recall Curve).	Falta de profundidade no que tange a ataques de backdoor mais complexos ou ofuscados, gerando falsos positivos.
Hong, Hanbin <i>et al.</i> [2]	Sistematização do conhecimento em segurança de prompts em LLMs, propondo uma taxonomia e um conjunto de métricas próprias para padronizar avaliações de ataques e defesas.	Revisão sistemática e proposta de uma taxonomia multi-nível. Desenvolvimento de toolkit aberto e dataset (JailbreakDB) para avaliação padronizada.	Propõe métricas padronizadas próprias integradas a um toolkit de avaliação com taxonomia hierárquica e perfis de ameaça formais.	Limitação na aplicação prática das métricas e taxonomias propostas; necessidade de validação contínua e ampliação para modelos multimodais e cenários reais.
Chen, Sizhe <i>et al.</i> [3]	Proposta de defesa chamada <i>StruQ</i> , que utiliza consultas estruturadas (<i>structured queries</i>) para separar dados do prompt, reduzindo o risco de injeções maliciosas em LLMs.	Implementação experimental com análise de desempenho e eficácia em diferentes cenários de injeção. Avalia a separação entre dados e instruções para mitigar ataques.	Taxa de sucesso de ataque, latência e sobrecarga do sistema (<i>defense overhead</i>).	Boa eficácia em prompts simples, mas limitação em ataques indiretos e cenários complexos de múltiplas etapas, exigindo integração com outras técnicas de defesa.
Benjamin, Victoria <i>et al.</i> [4]	Análise sistemática da vulnerabilidade de 36 LLMs a ataques de injeção de prompt focados em gerar código de keylogger.	4 prompts de injeção direta contra 36 LLMs. Análise estatística (Correlação, Random Forest, SHAP, PCA).	(1) Taxa de sucesso (2) Importância de features. (3) Correlação entre prompts.	Necessidade de testes multilíngues, múltiplas etapas de complexidade.
Sebastian, Glorin. [5]	Investigar a proteção de dados e privacidade em chatbots (foco no ChatGPT). Avaliar Tecnologias de Aprimoramento de Privacidade (PETs).	Revisão da literatura, análise de técnicas (ex: privacidade diferencial) e uma pesquisa (survey) com 177 usuários.	Métricas de percepção da pesquisa: (1) Nível de preocupação, (2) Disposição para sacrificar desempenho, (3) Consciência sobre vazamentos.	Análise de riscos à privacidade, preocupação dos usuários através de pesquisas.
This Work	Análise comparativa de frameworks de defesa contra injeção de prompt em LLMs.	Prova de conceito e simulações baseadas em diferentes cenários de ataque.	Taxa de sucesso de ataque, resiliência e custo computacional.	Busca preencher a lacuna de falta de metodologia padronizada para comparação empírica entre defesas.

IV. ARQUITETURA

A arquitetura proposta foi concebida como um sistema de segurança multicamadas para Modelos de Linguagem de Grande Escala (LLMs), estruturado com o objetivo de detectar, mitigar e bloquear ameaças semânticas e estruturais ao longo de todo o ciclo de processamento dos prompts. Essa organização modular distribui responsabilidades entre componentes especializados, permitindo robustez, escalabilidade e transparência no tratamento de riscos associados à interação com LLMs.

A Figura 1 apresenta uma visão geral da arquitetura, desta-

cando o fluxo de informações entre os módulos e os pontos de verificação de segurança aplicados em cada etapa.

A. Componentes Principais

A arquitetura é composta por quatro módulos centrais:

- 1) **Sanitizer**;
- 2) **Input Guardrail**;
- 3) **Bias and Output Guardrail**;
- 4) **Orchestrator Service**.

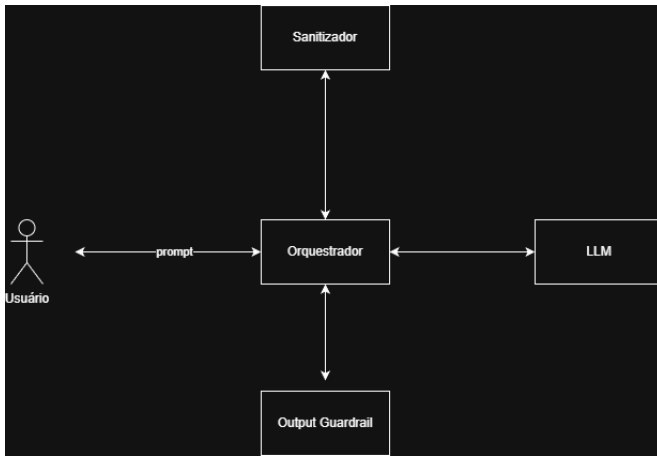


Fig. 1. Arquitetura geral do sistema de segurança proposto para LLMs.

Cada módulo atua de forma complementar, compondo um pipeline de análise capaz de interceptar ameaças explícitas, implícitas e semânticas.

B. Sanitizer

O *Sanitizer* constitui a primeira linha de defesa. Ele realiza o pré-processamento estrutural dos prompts, assegurando que o conteúdo enviado para as etapas seguintes esteja em formato consistente e livre de artefatos maliciosos.

Suas funções principais incluem:

- remoção de caracteres invisíveis e não padronizados;
- normalização e limpeza de formatos Unicode;
- detecção e neutralização de padrões de evasão e ofuscação;
- preservação do conteúdo semântico sempre que possível.

Ao padronizar a entrada, o módulo reduz a superfície de ataque e estabelece uma base confiável para as verificações subsequentes.

C. Input Guardrail

A segunda camada, o *Input Guardrail*, efetua verificações sintáticas e semânticas sobre o texto sanitizado. Seu papel é identificar riscos explícitos antes que o prompt seja encaminhado ao LLM.

Entre suas funções, destacam-se:

- identificação de tentativas de *prompt injection*;
- detecção de padrões de *jailbreak*;
- classificação de intenções maliciosas;
- detecção de informações pessoais identificáveis (PII);
- bloqueio imediato de entradas consideradas perigosas.

O módulo retorna uma resposta estruturada em caso de violação, garantindo que o LLM nunca receba um prompt inseguro.

D. Bias and Output Guardrail

A terceira camada é responsável por verificar tanto a entrada quanto a resposta gerada pelo LLM. Esse módulo combina regras sintáticas e classificações semânticas para detectar riscos mais complexos.

As funcionalidades incluem:

- detecção de vieses linguísticos em categorias como gênero, raça, idade, corpo, orientação ou classe social;
- identificação de alucinações (*hallucinations*) e inconsistências factuais;
- bloqueio de conteúdo proibido ou antiético;
- detecção de possíveis violações de PII após a geração da resposta.

A atuação pós-geração é essencial para impedir que o modelo devolva ao usuário conteúdo inadequado ou incorreto.

E. Orchestrator Service

O *Orchestrator* coordena todo o fluxo entre os módulos, garantindo que cada etapa seja executada no momento adequado e que todas as verificações sejam cumpridas.

Entre suas responsabilidades, destacam-se:

- encaminhar o prompt sanitizado para os módulos apropriados;
- decidir se o prompt deve ser processado pelo LLM com base nas verificações anteriores;
- avaliar a resposta gerada e aplicar verificações finais;
- compor o retorno ao usuário de forma padronizada, incluindo campos como `blocked`, `risk_category`, `sanitized` e `llm_response`.

Esse módulo abstrai a complexidade interna da arquitetura e garante interoperabilidade e auditabilidade.

F. Fluxo Operacional

O fluxo operacional completo pode ser descrito em seis etapas:

- 1) O usuário envia um prompt bruto.
- 2) O *Sanitizer* limpa e normaliza a entrada.
- 3) O *Input Guardrail* verifica se há riscos explícitos.
- 4) Caso aprovado, o prompt segue para o LLM.
- 5) A resposta gerada é analisada pelo *Bias and Output Guardrail*.
- 6) O *Orchestrator* compila o resultado final e retorna ao usuário.

Esse fluxo garante que tanto a entrada quanto a saída sejam avaliadas, configurando uma defesa integrada de ponta a ponta.

G. Justificativa Arquitetural

A adoção de uma arquitetura multicamadas fundamenta-se na diversidade de ameaças enfrentadas por sistemas baseados em LLMs. Variações nos tipos de ataque, na origem da interferência e na natureza dos riscos exigem um sistema capaz de operar em múltiplos níveis analíticos.

A arquitetura proposta:

- evita dependência de um único módulo para detecção de riscos;
- possibilita evolução modular com inclusão de novos mecanismos;
- assegura interpretabilidade e rastreabilidade das decisões;
- promove conformidade ética e redução de vieses prejudiciais.

Assim, a arquitetura estabelece uma base robusta para mecanismos avançados de segurança, sustentando os experimentos e análises discutidos na Seção VI.

V. MODELO PROPOSTO

Esta seção apresenta o modelo conceitual que fundamenta a arquitetura multicamadas desenvolvida para fortalecer a segurança de Modelos de Linguagem de Grande Escala (LLMs). Enquanto a seção de Arquitetura descreve a organização estrutural dos componentes, o modelo proposto formaliza os princípios, pressupostos e a lógica operacional que regem como a segurança é aplicada ao longo de todo o fluxo de interação.

A. Princípios de Projeto

O modelo proposto baseia-se em quatro princípios fundamentais:

- **Defesa em Camadas:** Nenhum mecanismo isolado é suficiente para garantir segurança em sistemas baseados em LLMs. Múltiplas camadas independentes aumentam a robustez contra diferentes vetores de ataque.
- **Filtragem Sensível ao Contexto:** Muitas ameaças e vieses se manifestam em nível semântico. Assim, o modelo incorpora verificações contextuais capazes de identificar riscos implícitos ou sutis.
- **Interferência Mínima:** O sistema deve intervir apenas quando necessário, preservando a fluidez da interação e reduzindo falsos positivos.
- **Aplicação Transparente:** Todas as decisões devem ser rastreáveis, auditáveis e interpretáveis, atendendo aos princípios de segurança e conformidade ética.

Esses princípios orientam a integração e o comportamento dos módulos que compõem a arquitetura.

B. Modelo de Ameaças

O modelo proposto assume que o sistema pode ser alvo das seguintes capacidades adversárias:

- Tentativas diretas ou indiretas de *prompt injection* para subverter instruções do sistema.
- Solicitações destinadas a induzir o modelo a gerar conteúdo nocivo, como código malicioso, discurso de ódio ou desinformação.
- Submissão de prompts enviesados com a intenção de provocar respostas discriminatórias.
- Tentativas de extrair ou manipular informações pessoais identificáveis (PII).
- Indução de alucinações explorando ambiguidades ou limitações do modelo.

A estratégia de defesa é, portanto, bifásica: monitora tanto a entrada (antes do processamento pelo LLM) quanto a saída (após a geração da resposta).

C. Visão Geral do Modelo Conceitual

O modelo organiza o processo de defesa em três camadas conceituais:

- 1) **Camada de Normalização Estrutural:** Responsável pela sanitização da entrada, removendo caracteres invisíveis, padrões de ofuscação e anomalias de codificação, garantindo um formato limpo e consistente.
- 2) **Camada de Identificação Semântica de Riscos:** Detecta intenções maliciosas, tentativas de *jailbreak*, presença de PII e categorias de vies por meio de classificadores baseados em regras e padrões semânticos.
- 3) **Camada de Validação Pós-Geração:** Examina a resposta produzida pelo LLM para identificar alucinações, conteúdo proibido, violações éticas ou vazamento de informações sensíveis.

As camadas operam de forma sequencial, mas mantêm independência lógica para garantir resiliência caso alguma verificação isolada falhe.

D. Lógica Operacional

O funcionamento do modelo segue o fluxo abaixo:

- 1) O usuário envia um prompt.
- 2) O *Sanitizer* aplica normalização estrutural e remove potenciais artefatos nocivos.
- 3) O *Input Guardrail* avalia o prompt sanitizado quanto a riscos explícitos.
- 4) Caso o prompt seja considerado seguro, ele é encaminhado ao LLM; caso contrário, o sistema bloqueia a requisição e retorna uma resposta padronizada.
- 5) O LLM gera uma resposta.
- 6) O *Bias and Output Guardrail* avalia a resposta quanto à presença de alucinações, vieses, conteúdo indevido e possíveis violações de segurança.
- 7) O *Orchestrator* compila um retorno estruturado contendo o status da verificação e a resposta final ou mensagem de bloqueio.

Esse fluxo estabelece um mecanismo de defesa fechado, atuando nos dois extremos da interação.

E. Vantagens do Modelo

O modelo proposto apresenta diversas vantagens, entre elas:

- **Alta Resiliência:** Camadas múltiplas asseguram que a detecção não dependa de um único ponto de falha.
- **Escalabilidade:** Novos módulos podem ser adicionados sem alterar a lógica central.
- **Transparência:** Respostas padronizadas permitem auditoria e rastreamento de decisões.
- **Conformidade Ética:** A detecção de vieses e alucinações reforça práticas de IA responsável.
- **Generalização:** O modelo é compatível com diferentes LLMs e APIs, exigindo apenas adaptações mínimas.

F. Síntese

O modelo proposto formaliza uma estratégia de defesa multicamadas, sensível ao contexto e orientada por princípios de segurança e ética. Ele estabelece a base conceitual para os experimentos, análises e resultados apresentados na Seção VI.

- **Categorias de viés:** tempos entre 2.8 e 3.4 segundos, indicando maior custo computacional para análise semântica.

A média geral de 0.508s, embora adequada no contexto experimental, sugere que otimizações no módulo de vieses podem reduzir significativamente a latência do sistema.

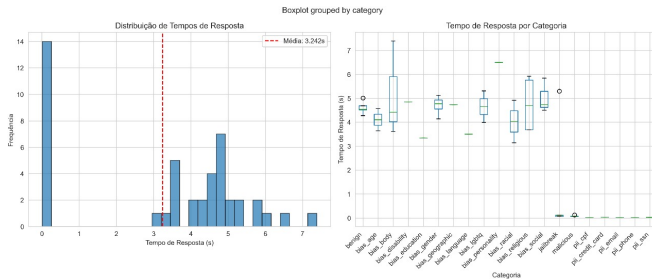


Fig. 5. Distribuição dos tempos de resposta por categoria.

E. Síntese dos Resultados

A análise global demonstra que:

- A arquitetura é altamente eficaz para ataques explícitos e identificação de PII.
- A detecção de vieses exige aprimoramentos significativos.
- Não houve falsos positivos, reforçando a preservação da experiência do usuário.
- Os tempos de resposta são consistentes e adequados, com oportunidades de otimização no Bias Guardrail.

Esses resultados validam a hipótese do trabalho: *uma arquitetura multicamadas melhora a segurança em LLMs*, embora módulos específicos (como o de viés) ainda precisem ser refinados.

VII. CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho apresentou uma arquitetura multicamadas voltada para a segurança de Modelos de Linguagem de Grande Escala (LLMs), integrando mecanismos de sanitização, detecção de riscos explícitos, identificação de vieses e validação pós-geração. Os experimentos realizados demonstraram que a abordagem é tecnicamente viável, resiliente e capaz de mitigar de forma eficaz uma variedade de ameaças associadas ao uso de LLMs em ambientes abertos.

O sistema mostrou desempenho robusto na detecção de ataques explícitos, incluindo tentativas de *prompt injection*, *jailbreak* e solicitações maliciosas, além de apresentar eficácia total em todas as categorias de informações pessoais identificáveis (PII). As métricas obtidas — com F1-score igual a 1.00 para categorias de risco explícito — validam a capacidade da arquitetura de atuar como um mecanismo confiável de defesa em camadas. A ausência de falsos positivos reforça a minimização de interferência indevida na experiência do usuário.

Por outro lado, os resultados também evidenciaram limitações importantes, sobretudo na detecção de vieses linguísticos implícitos. Categorias como gênero, corpo, educação e LGBTQIA+ permaneceram com baixa ou nula taxa de

sucesso, indicando que heurísticas baseadas apenas em regras e padrões simples não são suficientes para capturar nuances semânticas presentes em conteúdos enviesados. Essa limitação é consistente com desafios identificados na literatura e reforça a necessidade de incorporar mecanismos mais avançados de análise.

De forma geral, os resultados confirmam a hipótese central deste trabalho: *uma arquitetura multicamadas, sensível ao contexto e baseada em verificações estruturais e semânticas, aumenta significativamente a segurança e a confiabilidade de sistemas baseados em LLMs*. O modelo proposto estabelece um arcabouço sólido para futuras extensões e aprimoramentos em sistemas de IA responsáveis.

Trabalhos Futuros

Com base nas limitações observadas e nas oportunidades identificadas ao longo deste estudo, destacam-se as seguintes direções para trabalhos futuros:

- **Aprimoramento da detecção de vieses:** Investigar o uso de classificadores supervisionados, embeddings semânticos e modelos específicos para vieses, a fim de ampliar a cobertura e reduzir falsos negativos.
- **Otimização do Bias Guardrail:** Reduzir a latência do módulo por meio de técnicas como pré-filtragem, paralelização, cache semântico e modelos mais leves para análise textual.
- **Ampliação do conjunto de testes:** Construir um dataset mais extenso, diversificado e representativo, incluindo casos reais de ataques, vieses complexos e cenários ambíguos.
- **Integração com modelos de avaliação contextual:** Incorporar módulos capazes de analisar contexto discursivo, pragmático e situacional, permitindo identificar violações implícitas que escapam a regras estáticas.
- **Melhorias no contrato de resposta:** Expandir o esquema padronizado do *Orchestrator* para facilitar auditabilidade, rastreamento de decisões e integração com sistemas corporativos.
- **Ciclos contínuos de validação:** Adotar práticas de CI/CD com testes automáticos de segurança, avaliando regressões de desempenho conforme novos modelos ou atualizações são introduzidos.
- **Avaliação em cenários reais:** Implantar a arquitetura em contextos reais — como chatbots institucionais, sistemas educacionais ou plataformas de suporte — para observar comportamento sob demanda real e requisitos operacionais diversos.

Assim, este trabalho não apenas demonstra a viabilidade de mecanismos de segurança multicamadas para LLMs, mas também abre caminho para pesquisas futuras que possam ampliar a precisão, reduzir a latência, fortalecer a detecção de vieses e consolidar práticas de IA responsável em ambientes reais.

REFERENCES

- [1] H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao, “Uniguards: A unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models,” *ArXiv*, vol. abs/2502.13141, 2025.

- [2] H. Hong, S. Feng, N. Naderloui, S. Yan, J. Zhang, B. Liu, A. Arastehfard, H. Huang, and Y. Hong, "Sok taxonomy and evaluation of prompt security in large language models," *arXiv*, vol. abs/2510.15476, 2025.
- [3] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "Struq: Defending against prompt injection with structured queries," *USENIX Security Symposium*, 2025. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity25-chen-sizhe.pdf>
- [4] V. Benjamin, E. Braca, I. Carter, H. Kanchwala, N. Khojasteh, C. Landow, Y. Luo, C. Ma, A. Magarelli, R. Mirin, A. Moyer, K. Simpson, A. Skawinski, and T. Heverin, "Systematically analyzing prompt injection vulnerabilities in diverse llm architectures," *arXiv*, vol. abs/2410.23308, 2024.
- [5] G. Sebastian, "Privacy and data protection in chatgpt and other ai chatbots: Strategies for securing user information," *IJSPPC*, vol. 15, 2023.